

```
import os
import csv
import json
import uuid
from datetime import datetime
import yaml
import joblib
import numpy as np
import pandas as pd
from typing import Dict, Union, List, Any
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score, mean_absolute_error

def load_config(config_path="config.yaml"):
    if not os.path.exists(config_path):
        raise FileNotFoundError(f"Configuration file missing at: '{config_path}'")
    with open(config_path, "r") as f:
        return yaml.safe_load(f)

def _log_experiment_run(run_dict: dict, engine_name: str, model_params: dict,
model_uuid: str, cfg: dict,):
    csv_path = cfg["paths"]["experiment_log"]
    os.makedirs(os.path.dirname(csv_path), exist_ok=True)

    headers = [
        "timestamp", "model_uuid", "engine_type", "delta_threshold", "calculated_delta_r2",
        "statistical_outcome", "operational_outcome", "full_mean_cv_r2",
        "restricted_mean_cv_r2", "delta_std_cv_r2", "full_mean_cv_mae_km",
        "restricted_mean_cv_mae_km", "model_parameters"
    ]

    summary = run_dict["summary_averages"]
    hypotheses = run_dict["hypothesis_results"]

    row_data = {
        "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "model_uuid": model_uuid,
        "engine_type": engine_name.upper(),
        "delta_threshold": hypotheses["delta_threshold_used"],
        "calculated_delta_r2": hypotheses["calculated_delta_r2"],
        "statistical_outcome": hypotheses["statistical"]["outcome"],
```

```

"operational_outcome": hypotheses["operational"]["outcome"],
"full_mean_cv_r2": summary["full_mean_cv_r2"],
"restricted_mean_cv_r2": summary["restricted_mean_cv_r2"],
"delta_std_cv_r2": summary["delta_std_cv_r2"],
"full_mean_cv_mae_km": summary["full_mean_cv_mae_km"],
"restricted_mean_cv_mae_km": summary["restricted_mean_cv_mae_km"],
"model_parameters": json.dumps(model_params)
}

file_exists = os.path.exists(csv_path)
with open(csv_path, mode="a", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=headers)
    if not file_exists:
        writer.writeheader()
    writer.writerow(row_data)

print(f"History logged automatically under [{engine_name.upper()}] with UUID:
{model_uuid}")

def train_model(
    engine_type: str = "rf",
    dataset_cleaned_path: str = None,
    config_path: str = "config.yaml",
    override_params: Dict[str, Any] = None,
    override_x_full: List[str] = None,
    override_x_restricted: List[str] = None,
    delta_threshold: float = 0.05
) -> dict:

    cfg = load_config(config_path)

    final_data_path = dataset_cleaned_path if dataset_cleaned_path is not None else
    cfg["paths"]["dataset_cleaned"]
    if not os.path.exists(final_data_path):
        raise FileNotFoundError(f"Cleaned asteroid dataset not found at: '{final_data_path}'")

    engine_type = engine_type.lower()
    if engine_type == "rf":
        ModelRegressor = RandomForestRegressor
        param_key = "rf_params"
        model_name_label = "RANDOM FOREST"
    elif engine_type in ["xgboost", "xgb", "gradient"]:
        ModelRegressor = GradientBoostingRegressor
        param_key = "xgb_params"

```

```

    model_name_label = "XGBOOST"
else:
    raise ValueError("engine_type must be either 'rf' or 'xgboost'")

model_params = cfg["model_params"][param_key].copy()
if override_params is not None:
    model_params.update(override_params)

model_dir = cfg["paths"]["model_dir"]
os.makedirs(model_dir, exist_ok=True)

x_full = override_x_full if override_x_full is not None else cfg["features"]["x_full"]
x_restricted = override_x_restricted if override_x_restricted is not None else
cfg["features"]["x_restricted"]
target_col = cfg["features"]["target_col"]

df_model = pd.read_csv(final_data_path)
all_needed_features = list(set(x_full + x_restricted))
X = df_model[all_needed_features]
y = df_model[target_col].values.ravel()

# Setup Validation
print("\n" + "="*50)
print(f"INITIALIZING UNIFIED {model_name_label} PIPELINE")
print("="*50)
print(f"Dataset Path:  {final_data_path}")
print(f"Dataset Shape:  {df_model.shape}")
print(f"Full Features:  {x_full}")
print(f"Restrictive Features: {x_restricted}")
print(f"Target Column:  {target_col}")
print(f"Model Parameters: {model_params}")
print("="*50 + "\n")

kf = KFold(n_splits=5, shuffle=True, random_state=42)
metrics = {"full": {"r2": [], "mae_km": []}, "restricted": {"r2": [], "mae_km": []}, "delta": {"r2":
[]}}

print(f"Running {model_name_label} Cross-Validation Evaluation...")
for train_idx, test_idx in kf.split(X, y):
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]

    rf_f = ModelRegressor(**model_params)
    rf_r = ModelRegressor(**model_params)

```

```

rf_f.fit(X_train[x_full], y_train)
preds_f = rf_f.predict(X_test[x_full])

rf_r.fit(X_train[x_restricted], y_train)
preds_r = rf_r.predict(X_test[x_restricted])

f_r2 = r2_score(y_test, preds_f)
r_r2 = r2_score(y_test, preds_r)

metrics["full"]["r2"].append(f_r2)
metrics["restricted"]["r2"].append(r_r2)
metrics["delta"]["r2"].append(f_r2 - r_r2)

y_test_km = 10 ** y_test
metrics["full"]["mae_km"].append(mean_absolute_error(y_test_km, 10 ** preds_f))
metrics["restricted"]["mae_km"].append(mean_absolute_error(y_test_km, 10 **
preds_r))

current_fold = len(metrics["full"]["r2"])
print(f"Fold {current_fold}/5 Complete | "
      f"Full R2: {f_r2:.4f} | "
      f"Restricted R2: {r_r2:.4f} | "
      f"Δ R2: {f_r2 - r_r2:.4f}")

full_composer = ColumnTransformer(transformers=[("keep", "passthrough", x_full)],
remainder="drop")
restricted_composer = ColumnTransformer(transformers=[("keep", "passthrough",
x_restricted)], remainder="drop")

print(f"Running {model_name_label} production pipeline...")
production_pipeline_full = Pipeline([("selector", full_composer), ("regressor",
ModelRegressor(**model_params))])
production_pipeline_restricted = Pipeline([("selector",
restricted_composer), ("regressor", ModelRegressor(**model_params))])

production_pipeline_full.fit(X, y)
production_pipeline_restricted.fit(X, y)

mean_full_r2 = float(np.mean(metrics["full"]["r2"]))
mean_restricted_r2 = float(np.mean(metrics["restricted"]["r2"]))
mean_delta_cv_r2 = mean_full_r2 - mean_restricted_r2
mean_full_mae = float(np.mean(metrics["full"]["mae_km"]))
mean_restricted_mae = float(np.mean(metrics["restricted"]["mae_km"]))

```

```

# Cross-Validation Averages Summary
print("\n" + "="*50)
print(f"CROSS-VALIDATION SUMMARY AVERAGES ({model_name_label})")
print("="*50)
print(f"Full Model Mean R2: {mean_full_r2:.4f}")
print(f"Restricted Model Mean R2: {mean_restricted_r2:.4f}")
print(f"Mean  $\Delta$  R2 (Drop Impact): {mean_delta_cv_r2:.4f} ( $\pm$ 
{np.std(metrics['delta']['r2']):.4f})")
print(f"Full Model Mean MAE: {mean_full_mae:.4f} km")
print(f"Restricted Model Mean MAE:{mean_restricted_mae:.4f} km")
print("="*50 + "\n")
print(f"Training final production {model_name_label} pipelines on full dataset...")

if mean_delta_cv_r2 > 0:
    statistical_result = "Reject H0_stat: Group 2 features significantly improve out-of-
sample R2."
    statistical_outcome = "H1_stat"
else:
    statistical_result = "Fail to Reject H0_stat: Group 2 features do not improve out-of-
sample R2."
    statistical_outcome = "H0_stat"

if mean_delta_cv_r2 <= delta_threshold:
    operational_result = f"Reject H0_ops: Group 2 feature improvement is  $\leq$  delta
({delta_threshold}). Restricted subset is operationally sufficient."
    operational_outcome = "H1_ops"
else:
    operational_result = f"Fail to Reject H0_ops: Group 2 feature improvement is > delta
({delta_threshold}). Restricted subset is insufficient."
    operational_outcome = "H0_ops"

generated_uuid = str(uuid.uuid4())
def _generate_metadata_payload(fitted_pipeline, feature_names, description,
cv_metrics):
    underlying_regressor = fitted_pipeline.named_steps["regressor"]

    if hasattr(underlying_regressor, "feature_importances_"):
        importances = underlying_regressor.feature_importances_.tolist()
        imp_dict = dict(zip(feature_names, importances))
    else:
        imp_dict = {"info": "Use sklearn.inspection.permutation_importance for boosting
metrics"}

```

```

return {
    "model_metadata": {
        "model_uuid": generated_uuid,
        "engine_type": engine_type,
        "description": description,
        "target_variable": target_col,
        "prediction_space_transformation": "10 ** prediction",
        "hyperparameters": underlying_regressor.get_params(),
    },
    "features": {
        "input_features": feature_names,
        "feature_importance": imp_dict
    },
    "validation_metrics": {
        "mean_cv_r2": float(np.mean(cv_metrics["r2"])),
        "std_cv_r2": float(np.std(cv_metrics["r2"])),
        "mean_cv_mae_km": float(np.mean(cv_metrics["mae_km"])),
    },
    "hypothesis_testing": {
        "delta_threshold_used": delta_threshold,
        "mean_delta_cv_r2": mean_delta_cv_r2,
        "statistical_outcome": statistical_outcome,
        "operational_outcome": operational_outcome
    }
}

```

```

full_payload = _generate_metadata_payload(production_pipeline_full, x_full, "Full
model with H.", metrics["full"])
restricted_payload = _generate_metadata_payload(production_pipeline_restricted,
x_restricted, "Restricted model without H.", metrics["restricted"])

```

```

full_model_path = os.path.join(model_dir, f"{engine_type}_full_model.pkl")
restricted_model_path = os.path.join(model_dir,
f"{engine_type}_restricted_model.pkl")

```

```

joblib.dump({"model": production_pipeline_full, "metadata": full_payload},
full_model_path)
joblib.dump({"model": production_pipeline_restricted, "metadata":
restricted_payload}, restricted_model_path)
print(f"Success! {model_name_label} production bundles saved safely (Old binaries
overwritten).")

```

```

print("\n" + "="*23 + f" HYPOTHESIS TESTING REPORT ({model_name_label}) " + "="*22)
print(f"Testing against Operational Tolerance ( $\delta$ ) = {delta_threshold}")

```

```

print(f"Calculated Delta R2 (Drop Impact):      = {mean_delta_cv_r2:.4f}")
print("-" * 72)
print(f"Statistical Test: [{statistical_outcome}] -> {statistical_result}")
print(f"Operational Test: [{operational_outcome}] -> {operational_result}")
print("=" * 72 + "\n")

```

```

output_payload = {
    "status": "success",
    "model_uuid": generated_uuid,
    "model_paths": {
        "full_model": full_model_path,
        "restricted_model": restricted_model_path
    },
    "hypothesis_results": {
        "delta_threshold_used": delta_threshold,
        "calculated_delta_r2": mean_delta_cv_r2,
        "statistical": {
            "outcome": statistical_outcome,
            "conclusion": statistical_result
        },
        "operational": {
            "outcome": operational_outcome,
            "conclusion": operational_result
        }
    },
    "summary_averages": {
        "full_mean_cv_r2": mean_full_r2,
        "restricted_mean_cv_r2": mean_restricted_r2,
        "delta_std_cv_r2": float(np.std(metrics["delta"]["r2"])),
        "full_mean_cv_mae_km": mean_full_mae,
        "restricted_mean_cv_mae_km": mean_restricted_mae
    },
    "raw_folds": {
        "fold_ids": list(range(1, 6)),
        "full_r2_per_fold": [float(v) for v in metrics["full"]["r2"]],
        "restricted_r2_per_fold": [float(v) for v in metrics["restricted"]["r2"]],
        "delta_r2_per_fold": [float(v) for v in metrics["delta"]["r2"]],
        "full_mae_km_per_fold": [float(v) for v in metrics["full"]["mae_km"]],
        "restricted_mae_km_per_fold": [float(v) for v in metrics["restricted"]["mae_km"]]
    }
}

```

```

_log_experiment_run(run_dict=output_payload, engine_name=engine_type,
model_params=model_params, model_uuid=generated_uuid, cfg=cfg)

```

```

return output_payload

def predict_model(
    input_data: Union[dict, pd.DataFrame],
    engine_type: str = "rf",
    model_type: str = "full",
    config_path: str = "config.yaml"
) -> Union[float, List[float], dict]:

    cfg = load_config(config_path)

    engine_type = engine_type.lower()
    if engine_type not in ["rf", "xgboost", "xgb", "gradient"]:
        raise ValueError("engine_type must be either 'rf' or 'xgboost'")

    if model_type.lower() == "both":
        return {
            "full_model_prediction": predict_model(input_data, engine_type, "full",
            config_path),
            "restricted_model_prediction": predict_model(input_data, engine_type,
            "restricted", config_path)
        }

    if model_type.lower() not in ["full", "restricted"]:
        raise ValueError("model_type must be either 'full', 'restricted', or 'both'")

    model_dir = cfg["paths"]["model_dir"]
    filename = f"{engine_type}_{model_type.lower()}_model.pkl"
    model_path = os.path.join(model_dir, filename)

    if not os.path.exists(model_path):
        model_key = "full_model" if model_type.lower() == "full" else "restricted_model"
        if model_key in cfg["paths"]:
            model_path = cfg["paths"][model_key]

    if not os.path.exists(model_path):
        raise FileNotFoundError(f"Production model binary missing at '{model_path}'. Please
run training first.")

    artifacts = joblib.load(model_path)
    pipeline = artifacts["model"]
    metadata = artifacts["metadata"]

```

```

    active_binary_engine = metadata["model_metadata"].get("engine_type",
engine_type).lower()

    requested_norm = "xgboost" if engine_type in ["xgboost", "xgb", "gradient"] else "rf"
    binary_norm = "xgboost" if active_binary_engine in ["xgboost", "xgb", "gradient"] else "rf"

    if requested_norm != binary_norm:
        raise ValueError(
            f"Engine Mismatch Error!\n"
            f"The active model binary file on your disk belongs to a '{binary_norm.upper()}'
run.\n"
            f"However, your predict_model call requested a '{requested_norm.upper()}' run.\n"
            f>Please run train_model(engine_type='{engine_type}') to overwrite the production
binary first."
        )

    if isinstance(input_data, dict):
        df_input = pd.DataFrame([input_data])
    elif isinstance(input_data, pd.DataFrame):
        df_input = input_data.copy()
    else:
        raise TypeError("input_data must be a dictionary or a Pandas DataFrame.")

    log_prediction = pipeline.predict(df_input)
    diameter_km = 10 ** log_prediction

    if isinstance(input_data, dict):
        return diameter_km.item() if isinstance(diameter_km, np.ndarray) else
float(diameter_km)
    return diameter_km.tolist()

```